

Major US Power Outages

Name: Zack Mosley

Website Link: https://zmosds.github.io/predict_major_power_outages/

```
In [1]: import pandas as pd
import numpy as np
from pathlib import Path
```

```
import plotly.express as px
import plotly.io as pio
```

```
pd.options.plotting.backend = 'plotly'
pio.renderers.default = 'notebook'
```

```
# from dsc259r_utils import * # Feel free to uncomment and use this.
```

```
In [2]: # Load data and select relevant columns
cols = ['YEAR', 'MONTH', 'U.S._STATE', 'POSTAL.CODE', 'NERC.REGION', 'CLIMATE.CATEGORY', 'ANOMALY.LEVEL', 'CAUSE.CATEGORY', 'CAUSE.OUTAGE.START.DATE', 'OUTAGE.START.TIME', 'OUTAGE.RESTORATION.OUTAGE.DURATION', 'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW', 'TOTAL.CUSTOMERS', 'POPDEN_URBAN', 'PC.REALGSP.STATE', 'UTIL.C

df = pd.read_excel('../data/power_outages_us.xlsx', header=5, usecols=
df = df.reset_index(drop=True)
print("Data loaded.\n")
df.head(1)
```

Data loaded.

```
Out[2]:
```

	YEAR	MONTH	U.S._STATE	POSTAL.CODE	NERC.REGION	CLIMATE.REGION
0	2011	7.0	Minnesota	MN	MRO	East North Central

1 rows x 21 columns

Step 1: Introduction

Question:

What factors best predict how long a power outage will last?

Introduction

Power outages affect millions of Americans every year, disrupting daily life, businesses, and critical infrastructure. Understanding what drives outage duration could help utilities better prepare for and respond to future events. This project uses a dataset of major power outages in the United States from 2000 to 2016, containing 1534 rows and covering outage causes, climate conditions, geographic information, and economic data.

Column	Description
OUTAGE.DURATION	Duration of the outage in minutes (prediction target)
CAUSE.CATEGORY	High level category of the outage cause (e.g. severe weather, intentional attack)
CLIMATE.CATEGORY	Climate episode category during the outage (warm, cold, or normal)
U.S._STATE	State where the outage occurred
NERC.REGION	North American Electric Reliability Corporation region
ANOMALY.LEVEL	Oceanic El Nino/La Nina index indicating climate anomaly severity
CUSTOMERS.AFFECTED	Number of customers affected by the outage
DEMAND.LOSS.MW	Amount of peak demand lost in megawatts

Step 2: Data Cleaning and Exploratory Data Analysis

```
In [3]: # Cleaning pipeline

def drop_missing_start(df):
    # Drop rows missing start date/time -- can't construct timestamp
    df = df[df['OUTAGE.START.DATE'].notna() & df['OUTAGE.START.TIME'].notna()]
    return df

def combine_timestamps(df):
    # Combine start date and time into single timestamp
    df['OUTAGE.START'] = pd.to_datetime(
        df['OUTAGE.START.DATE'].astype(str) + ' ' + df['OUTAGE.START.TIME'].astype(str),
        format='mixed', errors='coerce')

    # Combine restoration date and time into single timestamp
    df['OUTAGE.RESTORATION'] = pd.to_datetime(
        df['OUTAGE.RESTORATION.DATE'].astype(str) + ' ' + df['OUTAGE.RESTORATION.TIME'].astype(str),
        format='mixed', errors='coerce')
```

```

# Drop the original separate columns
df = df.drop(columns=['OUTAGE.START.DATE', 'OUTAGE.START.TIME', 'OUTAGE.DURATION'])
return df

def drop_invalid_durations(df):
    # Drop rows where duration is missing or zero -- can't predict or
    df = df[df['OUTAGE.DURATION'].notna() & (df['OUTAGE.DURATION'] > 0)]
    return df

def drop_non_continental(df):
    # Hawaii and Alaska aren't part of the continental U.S. climate re
    df = df[~df['U.S.STATE'].isin(['Hawaii', 'Alaska'])]
    return df

```

```

In [4]: # Apply cleaning pipeline
df = (df
      .pipe(drop_missing_start)
      .pipe(drop_invalid_durations)
      .pipe(drop_non_continental)
      .pipe(combine_timestamps)
      )
print("Applied cleaning pipeline.\n")
print(df.shape)

```

Applied cleaning pipeline.

(1393, 19)

Univariate Analysis

```

In [5]: def plot_univariate(df):
    # 1. Distribution of outage duration (target variable)
    outage_duration = px.histogram(df, x='OUTAGE.DURATION',
                                   title='Distribution of Outage Durat
                                   labels={'OUTAGE.DURATION': 'Outage

    outage_duration.update_xaxes(range=[-1, 20000]) # Trim tail for be
    outage_duration.update_yaxes(title_text='# of Outages')

    outage_duration.write_image('../docs/plots/univariate/outage_durat
    outage_duration.show()

    # 2. Count of outages by cause
    cause_counts = df['CAUSE.CATEGORY'].value_counts().reset_index()
    cause_counts.columns = ['Cause', 'Count']

    outage_cause = px.bar(cause_counts, x='Cause', y='Count',
                          title='Power Outages by Cause',
                          labels={'Cause': 'Cause', 'Count': '# of Out

    outage_cause.update_layout(xaxis_tickangle=-35, showlegend=False)

```

```
outage_cause.write_image('../docs/plots/univariate/outage_cause_co
outage_cause.show()

# 3. Distribution of anomaly level
anomaly = px.histogram(df, x='ANOMALY.LEVEL',
                        title='Distribution of El Niño and La Niña
                        labels={'ANOMALY.LEVEL': 'El Niño / La Niña

anomaly.update_xaxes(tickvals=[-1, 0, 1.5],
                     ticktext=["La Niña (Cooler)", "Normal", "El N
anomaly.update_yaxes(title_text='# of Outages')

anomaly.write_image('../docs/plots/univariate/anomaly_level_dist.p
anomaly.show()

plot_univariate(df)
```



```
In [6]: def plot_bivariate(df):  
  
    # Average outage duration by cause  
    cause_duration = df.groupby('CAUSE.CATEGORY')['OUTAGE.DURATION'].m  
    cause_duration.columns = ['Cause', 'Mean Duration']  
    cause_duration['Mean Duration'] = cause_duration['Mean Duration']  
    cause_duration = cause_duration.sort_values('Mean Duration', ascen  
  
    fig1 = px.bar(cause_duration, x='Cause', y='Mean Duration', title=  
        labels={'Cause': 'Cause Category', 'Mean Duration': 'Average D  
    )  
  
    fig1.update_layout(xaxis_tickangle=-35)  
    fig1.write_image('../docs/plots/bivariate/duration_by_cause.png')  
    fig1.show()  
  
    # Outage duration vs customers affected (in hours)  
    fig2 = px.scatter(  
        df,
```

```

x='CUSTOMERS.AFFECTED',
y=df['OUTAGE.DURATION'] / 60,
title='Outage Duration vs Customers Affected',
labels={
    'CUSTOMERS.AFFECTED': 'Customers Affected',
    'y': 'Outage Duration (hours)'
}
)

fig2.write_image('../docs/plots/bivariate/duration_vs_customers.png')
fig2.show()

# Average outage duration by state (in hours)
state_duration = df.groupby('U.S._STATE')['OUTAGE.DURATION'].mean()
state_duration.columns = ['State', 'Mean Duration']
state_duration['Mean Duration'] = state_duration['Mean Duration']
state_duration = state_duration.sort_values('Mean Duration', ascending=True)

fig3 = px.bar(
    state_duration, x='State', y='Mean Duration',
    title='Top 15 States by Average Outage Duration',
    labels={'State': 'State', 'Mean Duration': 'Average Duration (hours)'},
)

fig3.update_layout(xaxis_tickangle=-35)
fig3.write_image('../docs/plots/bivariate/duration_by_state.png')
fig3.show()

# Outage duration by climate category (in hours)
fig4 = px.box(
    df,
    x='CLIMATE.CATEGORY',
    y=df['OUTAGE.DURATION'] / 60,
    title='Outage Duration by Climate Category',
    labels={
        'CLIMATE.CATEGORY': 'Climate Category',
        'y': 'Outage Duration (hours)'
    },
    category_orders={'CLIMATE.CATEGORY': ['cold', 'normal', 'warm']},
)

fig4.write_image('../docs/plots/bivariate/duration_by_climate.png')
fig4.show()

plot_bivariate(df)

```



```
In [7]: def plot_aggregation(df):  
  
    # Mean duration by Cause and Climate Category (hours)  
    cause_climate = (  
        df.groupby(['CAUSE.CATEGORY', 'CLIMATE.CATEGORY'])['OUTAGE.DUR'  
        .mean()  
        .reset_index()  
    )  
  
    cause_climate['Mean Duration'] = cause_climate['OUTAGE.DURATION']  
    cause_climate = cause_climate.drop(columns='OUTAGE.DURATION')  
  
    fig1 = px.bar(  
        cause_climate,  
        x='CAUSE.CATEGORY',  
        y='Mean Duration',  
        color='CLIMATE.CATEGORY',  
        title='Mean Outage Duration by Cause and Climate Category',  
        labels={  
            'CAUSE.CATEGORY': 'Cause Category',
```

```

        'CLIMATE.CATEGORY': 'Climate Category',
        'Mean Duration': 'Average Duration (hours)'
    },
    barmode='group'
)

fig1.update_layout(xaxis_tickangle=-35)
fig1.write_image('../docs/plots/aggregation/duration_by_cause_clim
fig1.show()

# Mean duration by NERC Region (hours)
region_duration = (
    df.groupby('NERC.REGION')['OUTAGE.DURATION']
    .mean()
    .reset_index()
)

region_duration['Mean Duration'] = region_duration['OUTAGE.DURATION']
region_duration = region_duration.drop(columns='OUTAGE.DURATION')
region_duration = region_duration.sort_values('Mean Duration', asc

region_map = {
    'MRO': 'MRO (Midwest)',
    'RFC': 'RFC (Mid-Atlantic)',
    'SERC': 'SERC (Southeast)',
    'TRE': 'TRE (Texas)',
    'WECC': 'WECC (West)',
    'NPCC': 'NPCC (Northeast)',
    'FRCC': 'FRCC (Florida)'
}

region_duration['NERC.REGION'] = region_duration['NERC.REGION'].ma

fig2 = px.bar(
    region_duration,
    x='NERC.REGION',
    y='Mean Duration',
    title='Mean Outage Duration by NERC Region',
    labels={
        'NERC.REGION': 'NERC Region (U.S. Power Grid Region)',
        'Mean Duration': 'Average Duration (hours)'
    }
)

fig2.update_layout(xaxis_tickangle=-35)
fig2.write_image('../docs/plots/aggregation/duration_by_region.png
fig2.show()

plot_aggregation(df)

```


Step 3: Assessment of Missingness

```
In [8]: # TODO
# Create missingness indicator
df['CUSTOMERS_MISSING'] = df['CUSTOMERS.AFFECTED'].isna()

def permutation_test(df, group_col, missing_col, n_perm=1000):
    observed = (
        df.groupby(group_col)[missing_col]
        .mean()
        .max() -
        df.groupby(group_col)[missing_col]
        .mean()
        .min()
    )

    perm_stats = []
```

```

for _ in range(n_perm):
    shuffled = df.copy()
    shuffled[missing_col] = np.random.permutation(shuffled[missing_col])

    stat = (
        shuffled.groupby(group_col)[missing_col]
        .mean()
        .max() -
        shuffled.groupby(group_col)[missing_col]
        .mean()
        .min()
    )

    perm_stats.append(stat)

p_value = np.mean(np.array(perm_stats) >= observed)

return observed, p_value

permutation_test(df, 'CAUSE.CATEGORY', 'CUSTOMERS_MISSING')
#If p-value is small (< 0.05), missingness depends on cause category.

```

Out[8]: (0.806005855888024, 0.0)

In [9]: *# Example Where It Likely Does NOT Depend*
permutation_test(df, 'MONTH', 'CUSTOMERS_MISSING')

Out[9]: (0.2931034482758621, 0.0)

Step 4: Hypothesis Testing

Null Hypothesis (H_0): The mean outage duration during abnormal climate conditions (warm or cold) is equal to the mean outage duration during normal conditions.

- $\mu_{\text{abnormal}} = \mu_{\text{normal}}$

Alternative Hypothesis (H_1): The mean outage duration during abnormal climate conditions is greater than the mean outage duration during normal conditions.

- $\mu_{\text{abnormal}} > \mu_{\text{normal}}$

Test Statistic The difference in mean outage duration (in hours) between abnormal and normal climate conditions.

- Mean duration (abnormal) – Mean duration (normal)

```
In [12]: #
def climate_duration_permutation_test(df, n_perm=5000):

    # Create abnormal vs normal grouping
    climate_group = df['CLIMATE.CATEGORY'].replace({
        'warm': 'abnormal',
        'cold': 'abnormal',
        'normal': 'normal'
    })

    duration_hours = df['OUTAGE.DURATION'] / 60

    # Observed statistic
    observed = (
        duration_hours[climate_group == 'abnormal'].mean()
        -
        duration_hours[climate_group == 'normal'].mean()
    )

    perm_stats = []

    for _ in range(n_perm):
        shuffled = np.random.permutation(climate_group.values)

        stat = (
            duration_hours[shuffled == 'abnormal'].mean()
            -
            duration_hours[shuffled == 'normal'].mean()
        )

        perm_stats.append(stat)

    perm_stats = np.array(perm_stats)

    # One-sided test (greater than)
    p_value = np.mean(perm_stats >= observed)

    return observed, p_value
```

```
In [13]: observed_stat, p_value = climate_duration_permutation_test(df)

observed_stat, p_value
```

```
Out[13]: (3.491106864406433, 0.2674)
```

Results

- Observed difference = 3.49 hours
- p-value = 0.2674
- Fail to reject H_0

Conclusion

Because the p-value (0.2688) is greater than 0.05, we fail to reject the null hypothesis.

Although outages during abnormal climate conditions lasted approximately 3.49 hours longer on average, this difference is not statistically significant. The observed difference could reasonably have occurred due to random variation.

Therefore, we do not find sufficient evidence to conclude that abnormal climate conditions lead to longer outage durations.

Step 5: Framing a Prediction Problem

Can we predict, using only information available at the time an outage begins, whether the outage will become severe (defined as lasting 12 or more hours)?

To ensure the model is realistic, only variables known at outage onset are used as predictors. Information that becomes available after the outage progresses (such as restoration time, total duration, or demand loss) is excluded to prevent data leakage.

The goal is to build a classification model that estimates the probability that an outage will be severe based solely on start-time information.

Step 6: Baseline Model

```
In [14]: def make_onset_dataset(df, severe_hours=12):

    df = df.copy()

    # Create target
    df['SEVERE_OUTAGE'] = (df['OUTAGE.DURATION'] / 60 >= severe_hours)

    # Use already-created timestamp
    df['START_HOUR'] = df['OUTAGE.START'].dt.hour
    df['START_DAYOFWEEK'] = df['OUTAGE.START'].dt.dayofweek
    df['START_QUARTER'] = df['OUTAGE.START'].dt.quarter

    feature_cols = [
        'YEAR', 'MONTH', 'START_HOUR', 'START_DAYOFWEEK', 'START_QUARTER',
        'U.S._STATE', 'POSTAL.CODE', 'NERC.REGION', 'CLIMATE.REGION',
        'CLIMATE.CATEGORY', 'ANOMALY.LEVEL',
        'CAUSE.CATEGORY', 'CAUSE.CATEGORY.DETAIL',
```

```

        'TOTAL.CUSTOMERS', 'POPDEN_URBAN', 'PC.REALGSP.STATE', 'UTIL.C
    ]

    # Drop leakage columns
    df = df.drop(columns=[
        'OUTAGE.RESTORATION',
        'OUTAGE.DURATION',
        'CUSTOMERS.AFFECTED',
        'DEMAND.LOSS.MW'
    ], errors='ignore')

    model_df = df[feature_cols + ['SEVERE_OUTAGE', 'OUTAGE.START']]

    return model_df

```

```

In [15]: from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, average_precision_s

def train_severe_model(model_df, cutoff_date):

    train_df = model_df[model_df['OUTAGE.START'] < cutoff_date].copy()
    test_df = model_df[model_df['OUTAGE.START'] >= cutoff_date].copy()

    y_train = train_df['SEVERE_OUTAGE']
    y_test = test_df['SEVERE_OUTAGE']

    X_train = train_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])
    X_test = test_df.drop(columns=['SEVERE_OUTAGE', 'OUTAGE.START'])

    cat_cols = X_train.select_dtypes(include=['object']).columns.tolist()
    num_cols = X_train.select_dtypes(exclude=['object']).columns.tolist()

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', Pipeline(steps=[
                ('imputer', SimpleImputer(strategy='median')),
            ]), num_cols),
            ('cat', Pipeline(steps=[
                ('imputer', SimpleImputer(strategy='most_frequent')),
                ('onehot', OneHotEncoder(handle_unknown='ignore')),
            ]), cat_cols),
        ]
    )

    model = Pipeline(steps=[
        ('prep', preprocessor),
        ('clf', LogisticRegression(max_iter=2000, class_weight='balanc

```

```

])

model.fit(X_train, y_train)

preds = model.predict(X_test)
proba = model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, preds, digits=3))
print("Average Precision (PR AUC):", round(average_precision_score

return model

```

```

In [16]: model_df = make_onset_dataset(df, severe_hours=12)
model = train_severe_model(model_df, cutoff_date="2015-01-01")

```

	precision	recall	f1-score	support
0	0.764	0.802	0.783	101
1	0.565	0.510	0.536	51
accuracy			0.704	152
macro avg	0.665	0.656	0.659	152
weighted avg	0.697	0.704	0.700	152

Average Precision (PR AUC): 0.6014

/var/folders/ff/b7ds788x3bb79hzybxn8_qjw0000gn/T/ipykernel_25823/600961730.py:20: Pandas4Warning:

For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning. See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

Step 7: Final Model

```

In [8]: # TODO

```

Step 8: Fairness Analysis

```

In [9]: # TODO

```